L13B: The Last Lecture

CS1101S: Programming Methodology

Boyd Anderson, Sanka Rasnayaka, Martin Henz

13th November, 2026

Version 1.0 — Built: 2026-11-12 23:57:12 SGT

Outline

Wrapping up

Big Ideas of CS1101S

Python §4 vs Real Python

Areas of Computer Science

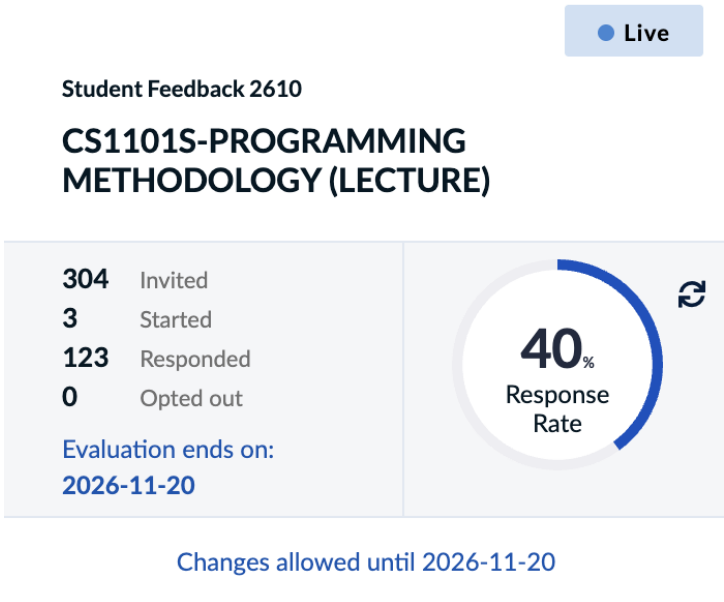
Goodbye?

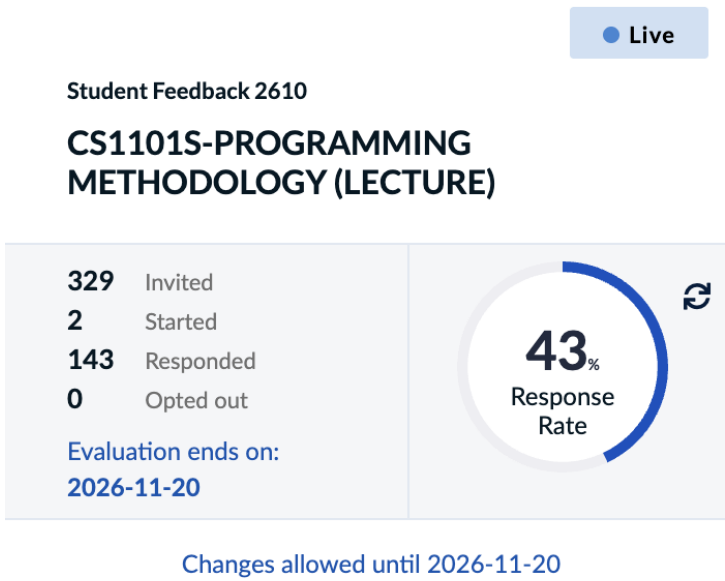
Some loose ends

- Source Academy XP: new threshold of **22,000 XP** for the full 5% (Quests gave less XP this semester); with $Y < 22{,}000$ XP you get $5\% \times Y/22{,}000$
- Mastery Checks: MC1 85%, MC2 36% (Avengers' discretion to entertain your late requests)
- Course feedback! **your link to give CS1101S feedback**

Course feedback response rate still low

Please give us your constructive feedback :)





Avengers needed: AY2027/28



Applications are open: fill this form

Deadline: Wednesday of Exam Week 2, 02/12/2026, 23:59

Please consider joining next year's team!

Source Academy

- Help improve Source Academy: join **CP3108** in Semester 2
- Offered only to first-year students who have taken CS1101S: work with the Source Academy team on new features and improvements
- Project ideas: the CP3108 GitHub project (click the title)
- Apply with this form by **tonight**, Friday 13/11/2026, 23:59
- Shortlisted applicants are interviewed in Reading Week or Exam Week 1 or 2

Final Assessment

When and where

25-Nov-2026, Wednesday, 9:00 AM – 11:00 AM (Duration: 2 hours), Venue: MPSH1-A/B

Scope of Final Assessment

All core course material: Lectures, Paths, Reflections, Studios, Missions

Outline

Wrapping up

Big Ideas of CS1101S

Python §4 vs Real Python

Areas of Computer Science

Goodbye?

Scope of names

- Lexical (static) scoping is the dominant scoping mechanism in modern programming languages
- Name occurrences refer to the closest surrounding declaration
- Lexical scoping affords readable and modular programs because the program text indicates what declarations the names refer to.
- `global` and `nonlocal` prevent local declaration

Beyond Python: Naming

How do you declare names? What is their visibility (scope)?

Functional programming

- Substitution model
- Functions are "first-class values"
- Elegant abstractions: map, reduce, filter
- Streams: lazy linked lists
- Lazy evaluation: variant of normal-order reduction

Beyond CS1101S: Less is more

Remain stateless, if possible (for example spreadsheets)
expressive power vs complexity

Iterative and recursive processes

- Programs describe computational processes
- Processes require resources (time and space)
- We use "order of growth" notation (O, Θ, Ω) to assess the resource requirements of processes as the given problems grow.
- Space is consumed by functions, pairs, lists and deferred operations.
- Iterative processes do not give rise to deferred operations.
- CPS guarantees iterative process, but not constant space consumption.

Beyond Python

What consumes time and space, and how much?

Environment model for program execution

- The simplest model of excution is the substitution model
- In the presence of state (assignment), the substitution model ceases to work
- The environment model provides a clear explanation of the execution of programs with lexical scoping and state.

Beyond Python

What is the right mental model to understand what happens when programs in language x run?

Mental models

1. Substitution model
2. Environment model
3. Metacircular evaluator in Python

Beyond CS1101S: Models are simplification devices

Everything should be made as simple as possible, but not simpler. (Albert Einstein)

The magic of computer science

We get to invent our own realities, and build mental models to understand these realities.

Data

- Pairs: simplest possible compound data structure
- Linked lists: recursive data structure, built from pairs
- Lists: index/value mapping, indices are numbers from $-$ length to length $-$ 1

Beyond Python §4

What data do you have, and what operations to access and change the data?

Memoization

- Remembering the result of function calls is a powerful programming technique
- Examples vary from catching flies to optimized streams

Beyond CS1101S

Bottom-up: dynamic programming, or top-down: tabling, memoization

Types in programs

- Types let us communicate clearly what data is involved in our computational processes
- Typed programming languages impose a type discipline on our programs
- Typed programming languages prevent errors before programs are shipped and run

Beyond Python §4 Typed

Type inference, dependent types, ...

Programming

- Communicating computational processes
- Five Step Method: Read, Play, Think, Program, Test

Beyond Python and CS1101S

Programming in-the-large: object-oriented programming, software engineering

Outline

Wrapping up

Big Ideas of CS1101S

Python §4 vs Real Python

Areas of Computer Science

Goodbye?

Python §4 vs Real Python

Python §4 is a *subset* of real Python: what you have learned carries over.

```
def sum_even_squares(xs):
    return reduce(lambda x, y: x + y, 0,
        map(lambda x: x * x,
            filter(lambda x: x % 2 == 0, xs)))

print(sum_even_squares(llist(1, 2, 3, 4)))
```



To run your programs outside Source Academy, do:
`pip install sourceacademy-sicp`

```
# Put this at the top of your program

from sicp import *
```

Python §4 vs Real Python: major differences

- Python allows any value as predicate:
  falsy are `False`, `None`, `0`, `[]`, `''`.
- Python does not specify efficient handling of what we called "iterative processes"
- General shape of `for` loops uses iterables:

```
for x in <iterable>
    ...
else:
    ...
```

`else` runs only if the loop finished without hitting break.

Python §4 vs Real Python: small conveniences

- Augmented assignment: `i += x` stands for `i = i + x` (also `-=`, `*=`, `/=`)
- Assignment to several names at once: `a, b = b, a` swaps two values
- Keyword parameters with default values: `def greet(name="world"):`
- f-strings: `f"value is {x}"` instead of `"value is " + str(x)`

Python §4 vs Real Python: more data structures

- Lists come with methods: `xs.append(5)`, `xs.pop()`, `xs.sort()`
- Slices: `xs[1:3]`, and `xs[::-1]` reverses a list
- Tuples: `(1, "one")`, fixed-size and immutable
- Dictionaries: `{"apple": 3, "pear": 5}`, a mapping from keys to values
- Sets: `{1, 2, 3}`, with fast membership tests
- Comprehensions: `[x * 2 for x in range(10)]` instead of `map` and `filter`

Python §4 vs Real Python: bigger programs

- Classes and objects (`self`, `__init__`, inheritance): previewed in Lecture L13A
- Exceptions: `try`/`except` and `raise`, instead of programs simply stopping with an error
- Generators (`yield`) for lazy sequences, instead of streams built from `pair` and `lambda`
- Modules and packages: `import numpy as np`
- Type hints: Python §4 Typed checks them before your program runs; real Python ignores them at run time, and separate tools (such as pyright) check them

Python §4 vs Real Python: where to go from here

- Install Python on your own machine, or try a Jupyter notebook
- Work through the official Python tutorial
- Popular libraries: NumPy, pandas, matplotlib, PyTorch
- Best of all: find a worthy project idea, and learn what you need to realize it

Outline

Wrapping up

Big Ideas of CS1101S

Python §4 vs Real Python

Areas of Computer Science

Goodbye?

Areas CS1101S touched on

Programming the whole course	**CS2030S** Programming Methodology II
Algorithms orders of growth, sorting, memoization	**CS2040S** Data Structures and Algorithms **CS3230** Design and Analysis of Algorithms **CS3233** Competitive Programming **CS4231** Parallel and Distributed Algorithms
Programming languages Python §1–§4, the metacircular evaluator	**CS2104** Programming Language Concepts **CS4212** Compiler Design **CS4215** Programming Language Implementation

Areas CS1101S touched on

Software engineering the Source Academy	**CS2103/T** Software Engineering **CP3108** Independent Project **CS3216, CS3281/2** Projects **CS4218** Software Testing
Computer graphics runes, curves, pix-n-flix	**CS3241** Computer Graphics **CS4247** Graphics Rendering Techniques

Areas CS1101S touched on

Robotics the Sumobot competition	**CS4278** Intelligent Robots
Games	**CS3247** Game Development **CS4350** Game Development Project **CS4246** Reinforcement Learning and Sequential Decision Mak

Areas CS1101S did not touch

Computer security	**CS2107** Introduction to Information Security **CS3235** Computer Security **CS4236** Cryptography in Practice: Systems and Applications **CS4238** Computer Security Practice **CS4239** Software Security
Computer architecture	**CS2100** Computer Organisation **CS4223** Multi-core Architectures
Operating systems	**CS2106** Introduction to Operating Systems **CG2271** Real-Time Operating Systems

Areas CS1101S did not touch

Databases	**CS2102** Database Systems **CS3223** Database Systems Implementation **CS4221** Database Applications Design and Tuning **CS4224** Distributed Databases
Networks	**CS2105** Introduction to Computer Networks **CS3103** Computer Networks Practice **CS4222** Wireless Networking **CS4226** Internet Architecture
Parallel and distributed computing	**CS3210** Parallel Computing **CS3211** Concurrent Programming

Areas CS1101S did not touch

Logic and formal methods	**CS3234** Logic for Proofs and Programs **CS4211** Formal Methods for Software Engineering
Theory	**CS3231** Theory of Computation **CS3236** Introduction to Information Theory
Human-computer interaction	**CS3249** Conversational User Interface **CS4240** Interaction Design for VR and AR
Machine learning	**CS2109S** Introduction to AI and Machine Learning **CS3244** Machine Learning
Computational biology	**CS2220** Introduction to Computational Biology

Outline

Wrapping up

Big Ideas of CS1101S

Python §4 vs Real Python

Areas of Computer Science

Goodbye?

Goodbye to CS1101S



Time to hang up our pink shirts!

How to do well in the final...

...and in computer science

Keep **I**t **S**imple, **S**tupid (the KISS principle)

Perfection is attained not when there is nothing more to add, but when there is nothing more to remove. A. de Saint-Exupéry

Full marks

In the past: your solution is correct and reasonably efficient

From now on: your solution is correct, reasonably efficient and does a good job at **communicating computational processes**

So, what was CS1101S?

We wanted you to:

- Understand
  the *structure and interpretation of computer programs*
- Learn how to program
- Learn computational thinking
- Get glimpses into the subject areas of computer science
- Get a feeling for how computer science "ticks"
- Fall in love with computer science

Thank you to the Source Academy Team

Zhang Yao, Source Academy CTO

Vemulapalli Akshay: Backend

Jain Shrey, Malani Aarav Sajeel: Frontend, Plugins, Modules, Evaluators

and the whole team!

Thank you to our Reflection Tutors and our co-instructors!

Eldric Liew Chun Tze, Enzio Kam Hai Hong, Zhang Yao

Martin Henz, Sanka Rasnayaka

Thank you to our Avengers!

The 2026 team



Check-In time!

Look for Check-In I13B in the Source Academy!

Just kidding! There is no check-in today.

Thank you!

We hope you had a great semester and wish you luck on all of your assessments and exams!